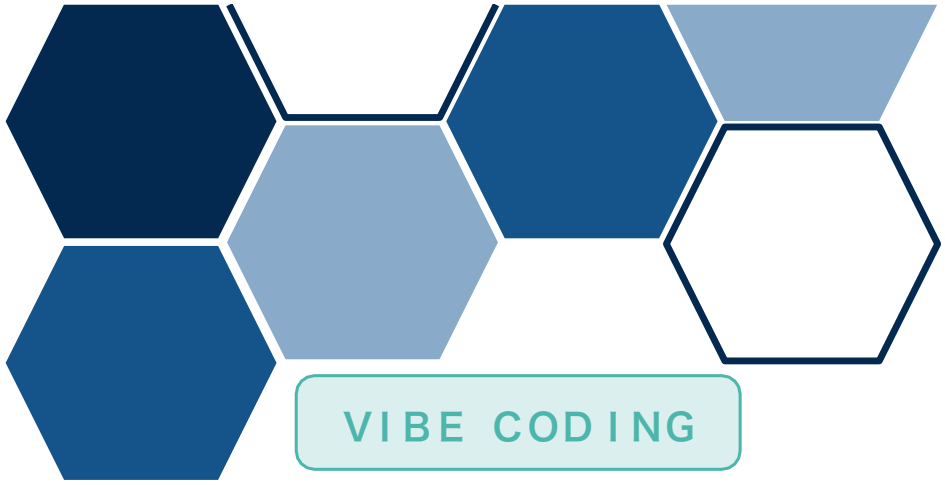


生成式软件开发

生成式软件开发-8

- 自动化测试

大纲

01 软件测试与质量
From Bug Hunting to Confidence Building

02 测试分层与策略
Test Pyramid, Cost and Testability

03 用户故事与验收标准
Making Requirements Verifiable

04 自动化验收测试的实现
From Stories to Executable Specifications

05 人机协同测试与开发工作流
Human-AI Collaborative Testing Workflow

06 界面自动化测试入门
UI Automation with Playwright

07 练习与讨论
Exercise & Discussion

01

软件测试与质量

软件测试与自动化测试的基本概念

基本概念与常用术语

缺陷（Defect / Bug）

代码/设计中的问题（隐患）。

失效/故障（Failure / Fault）

缺陷被触发并表现为用户可见的错误行为。

回归（Regression）

新改动导致已有的功能被破坏（旧 Bug 再次出现）。

不稳定测试（Flaky Test）

同一代码多次运行，结果时而成功、时而失败（非常消耗团队信任）。

测试用例（Test Case）

输入 + 操作 + 预期结果（测试 Oracle）。

验证（Verification）：我们是否把产品"按规格做对了"（build the thing right）？

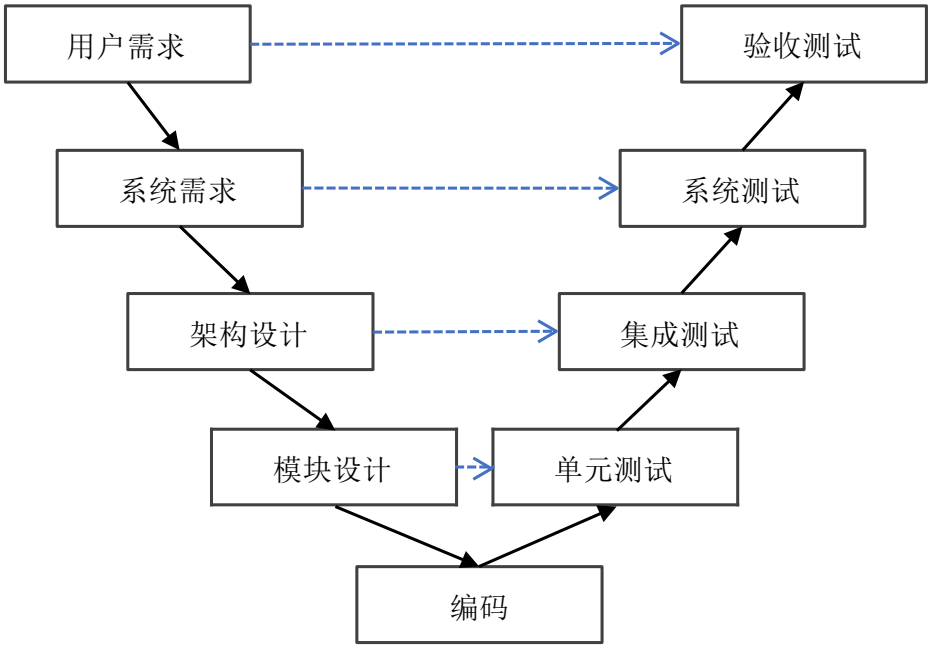
示例：检查代码是否精准实现了文档中定义的规则。自动化测试主要解决验证问题。

确认（Validation）：我们是否做了"正确的产品"（build the right thing）？

示例：虽然代码实现了 8 折优惠，但用户其实想要"满 100 减 20"，那么产品在"确认"层面就是失败的。这是人的核心职责。

测试在软件开发中的位置：V 模型

- “ 测试”不只是“ 找 Bug”。
- 在软件工程中，测试的核心作用是：**对照着规格说明(spec)，来验证代码是否按预期工作。**



V 模型：传统瀑布式方式中测试在开发过程中的位置

传统的 V 模型代表了“测试后置”的方法

- **先完成左边的开发流程**

  从需求开始，一步一步完成设计和编码

- **再逐步针对实现的内容进行测试**

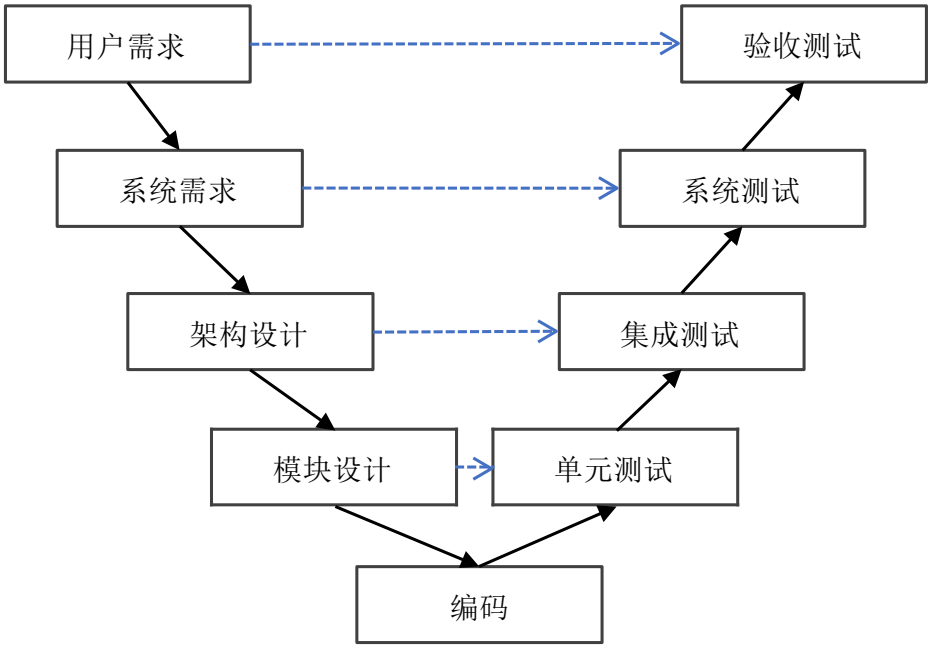
  针对编码内容，逐层自底向上开展测试。

核心洞察

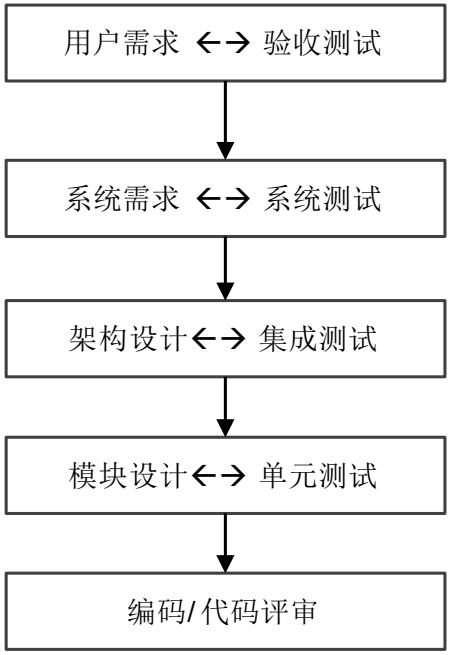
- 高层的测试（如验收测试）更关注用户侧的需求，低层的测试（如单元测试）更关注具体的代码实现
- 越高层的整体设计、用户需求，能测试的时间点越晚

测试在软件开发中的位置：从 V 模型 到 I 模型

- “ 测试” 不只是 “ 找 Bug”。
- 在软件工程中，测试的核心作用是：**对照着规格说明(spec)，来验证代码是否按预期工作。**



V 模型：传统瀑布式方式中测试在开发过程中的位置



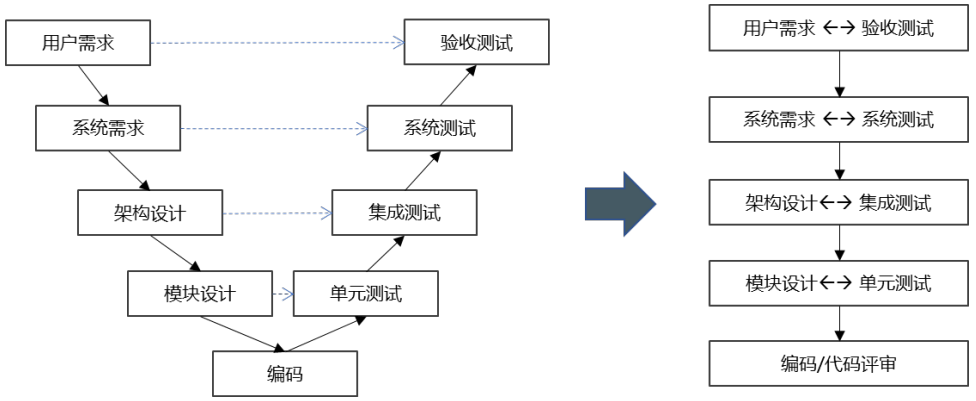
I 模型：测试前置的开发方式

- **测试前置**

  以终为始：按照最终验收需要的结果来指导开发过程，并且人有机会在第一时间确认是否最终就要这样

张刚，软件设计：从专业到卓越，人民邮电出版社，2022

测试与需求的关系



这样的**测试过程模型**揭示这样的信息：

1. 需求追踪

每一层测试来自其对应的“需求”或者“设计”。

2. 测试即需求的验证

测试不是在写完代码后盲目地"点点看"，而是根据你在左侧定义的规则（如用户故事、设计文档）进行**自动化审计**。

3. 把握需求可以从把握测试开始

不管是人还是AI，都需要在“要做什么”上建立其明确的认知，尽管这个认知会随着时间发生变化和演进。

核心洞察

你对需求的定义越清晰（可验证），你越能写出准确的测试；反之亦然。

高层的测试（如验收测试）更关注功能需求，低层的测试（如单元测试）更关注代码实现。

自动化测试

什么是自动化测试?

自动化测试是指通过计算机程序自动执行测试用例，来验证软件功能是否符合预期。通常由测试工具或测试框架支持，如 PlayWright、JUnit、pytest 等。

自动化测试在日常开发中主要解决三类问题

1. 快速反馈

改完代码马上知道"有没有把已有功能弄坏"。

2. 可沟通的规格

对于验收测试，就是把需求写成可执行的样子，让团队间、人与 AI 间的"理解一致"可被机器验证。

3. 可持续的质量保障

把关键功能固化成回归套件，降低回归测试的成本。

好的测试代码（作为评审清单）

01
确定性（Deterministic）

同样输入必然得到同样结果，不依赖随机与时间。测试必须是可重复的，任何不确定性都会侵蚀团队对测试的信任。

02
隔离性（Isolated）

尽量减少对外部系统（网络、真实数据库、真实时间）的依赖。使用替身（Stub/Mock）替代外部依赖，确保测试独立运行。

03
可读性（Readable）

失败时能让人立刻看懂"哪条需求没满足"。测试名称应体现领域语义，断言信息应清晰指出期望与实际值的差异。

04
可维护性（Maintainable）

需求变了，能以局部修改适配，而不是整体推倒重写。避免与实现细节强耦合，优先断言可观察的行为结果。

不好的测试代码（低价值测试）

反过来，很多测试写起来"看似覆盖了代码"，但价值极低：

✗ 断言过少或只断言"没有抛出异常"

测试只证明代码"跑完了"，但没有验证关键的业务约束和边界条件。

✗ 依赖外部不确定的输入，通过硬等固定时间获取结果

导致极易出现不稳定测试（Flaky Test），难以建立对系统功能正确的信心。

✗ 与界面实现细节强耦合

例如强依赖复杂的 DOM 层级结构。一旦页面结构变化，测试就要重新写。

我们将通过具体的代码对比，来看看什么是"好"与"坏"。

典型反例对比：断言过少

❌ **坏例子（只证明"代码跑完了"）**

```
def test_place_order_out_of_stock():
    svc = OrderService(
            inventory=FakeInventoryService(
                    {"SKU-1001": 1}),
            payment=SpyPaymentGateway())
    result = svc.place_order(
            sku="SKU-1001",
            qty=2,
            amount_cents=1999)
    # 只要状态是"拒绝"就行
    assert result["status"] == "REJECTED"
```

✅ **更好的写法（边界全写进断言）**

```
def test_reserve_seat_conflict():
    notification = NotificationServiceStub()
    svc = ReservationService(
            seat_repo=FakeSeatRepository(
            {"Seat-101": "AVAILABLE"}),
            notification=notification)
    result_a = svc.reserve("Seat-101", "User-A")
    result_b = svc.reserve("Seat-101", "User-B")
    assert result_b == {"status": "REJECTED",
            "reason": "座位已被占用"}
    assert "User-B" not in [n.user_id
            for n in notification.sent_messages]
```

缺陷分析：坏例子只验证了"状态是拒绝"，但遗漏了拒绝原因是否明确、是否错误发送了通知。

改进要点：把验收标准里的所有边界条件都写进断言，不仅验证"发生了什么"，还要验证"没发生什么"。

典型反例对比：硬等时间导致不稳定

❌ **坏例子（用固定时间赌系统刚好完成）**

```
await page.getByRole('button', { name: '确认' }).click();
// 硬等 3 秒。网络快了浪费时间，网络慢了测试直接报错
await page.waitForTimeout(3000);
// textContent() 不具备自动等待能力，元素没出现时返回
null 导致报错
const text = await page.locator('.status-message')
.textContent();
expect(text).toContain('预约成功');
```

✅ **更好的写法（等待"可观察条件"）**

```
await page.getByRole('button', {
name: '确认' }).click();
// 一直等到“ 预约成功” 出现为止，测试框架会自动帮我们等
await expect(page.getByText('预约成功')).toBeVisible();
// 进一步验证页面已跳转
await expect(page).toHaveURL(
/\/reservations\/\w+/);
```

核心原则

- 可以await expect就不要使用等待固定时间的 sleep / waitForTimeout
- 使用 expect().to...() 来等待可观察状态（文本出现、URL 变化、元素可见）
- 自动化UI测试工具（例如这里的PlayWright）具备自动重试与等待能力，直到条件满足或超时

如果要依赖外部不可靠的输入怎么办？ 通过构造测试替身的方式，让测试聚焦内部逻辑，而非外部依赖（后续会讲到）

典型反例对比：与实现细节强耦合

测试"怎么做"而不是"做到了什么"，一旦重构测试就全碎了。

❌ **坏例子（窥探"内部零件"）**

```
# Python：内部列表长度，一旦改用
# 数据库存储就会失败
assert len(svc._internal_reservations) == 3
assert svc._internal_reservations[0].state \
== "CONFIRMED“


// TS：页面结构稍微变一下，这个路径就找不到了
await page.locator(
'.reserve-form > div:nth-child(2) > button').click();
```

✅ **更好的写法（稳定的可观察结果）**

```
# Python：验证返回给用户的状态
result = svc.place_order(sku="SKU-1001",
qty=1, amount_cents=1999)
assert result["status"] == "PENDING_PAYMENT“


// TS：模拟真实用户，通过功能名称来寻找按钮
await page.getByRole('button', {name: '确认预约' }).click();
```

核心原则

- 只断言公共接口返回的可观察结果，不窥探私有状态
- UI 测试优先使用语义定位器（getByRole / getByLabel / getByText），而非 CSS 路径
- 测试应该站在"用户视角"验证"做到了什么"，而非"开发者视角"验证"怎么做的"

断言（Assertion）：代码里的"质检员"

在测试代码中，你会频繁看到 assert 或 expect 这样的词，它们被称为**断言**。

什么是断言？

断言是一个逻辑声明，它表达了你对程序在某一时刻状态的**期望**。简单来说，它就是在代码里理直气壮地宣称："此时此刻，结果**必须**是这样！"

为什么需要断言？

1. 自动审计：代替肉眼观察。快速验证实际运行结果是否与预期一致，比如值是否相等、是否抛出指定的异常等。

2. 帮助确定错误位置：如果运行结果不符合预期（如同一座位被分配给两个人），断言会立刻让程序"停下来"，并报告错误位置。在测试代码中必须使用。

计算机处理断言的动态过程

当代码执行到 assert 时，发生了什么？

1. 期望检查

自动核对 **实际结果（Actual）** 是否符合 **预期目标（Expected）**。

2. 静默与中断

- **符合预期**：程序继续执行下一行代码。
- **不符合预期**：立刻抛出异常 (Exception)，强制停止当前执行路径。

3. 自动化报告

测试框架（如 pytest 或 JUnit）捕捉异常，自动生成详尽的**差异报告**，精确指出哪一行代码、期望得到 A 却得到了 B。

测试代码和断言的演示

一个简单的计算器程序。现在要做一个对给定数值“翻倍”后记录的操作，对应的测试是怎样的？

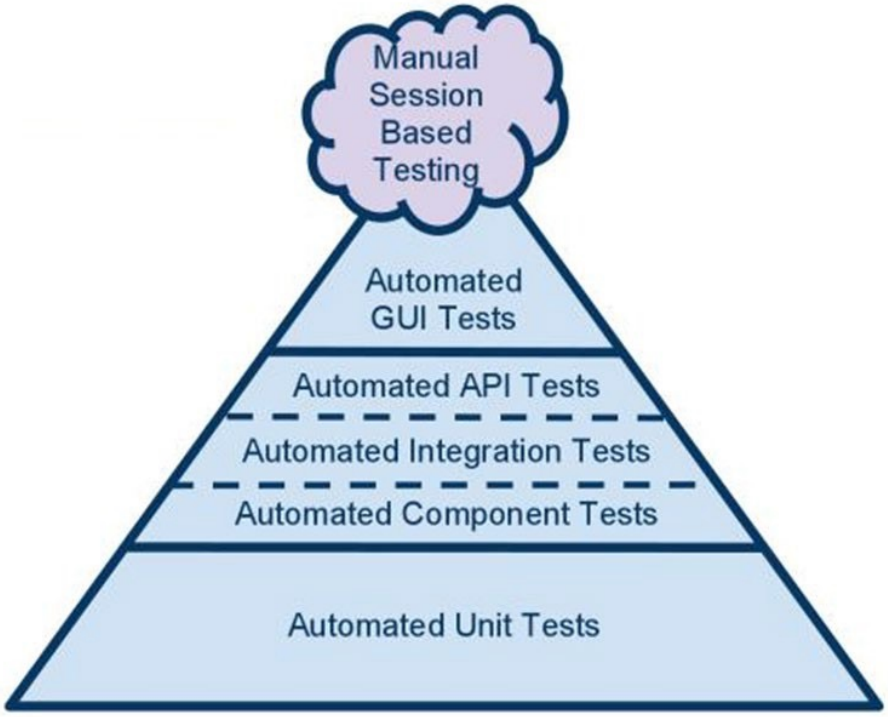


02

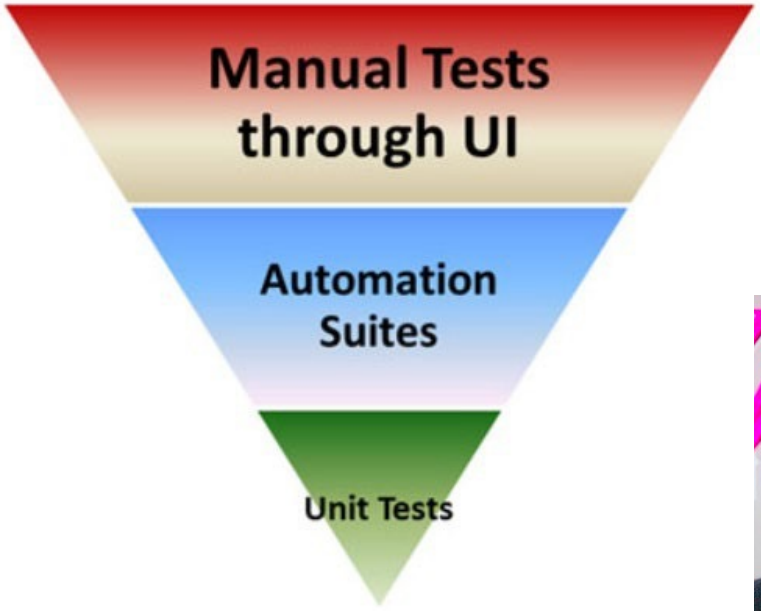
测试分层与策略

Test Pyramid, Cost and Testability

测试金字塔：分层测试代码的规模分布



理想的自动化测试金字塔

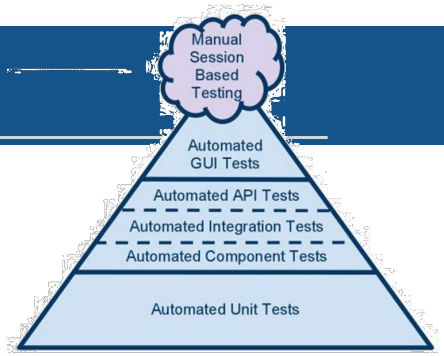


非理想的自动化测试倒金字塔



Martin Fowler

为什么不平均分配?



测试数量成金字塔形状分布的考虑:

1. 反馈速度

底层的单元测试运行快，能快速反馈；上层测试链路长、依赖多，速度慢。在增量功能的开发中，需要快速测试来验证已有的功能没有被改坏。

2. 隔离问题的层次

单元测试失败能指出模块局部的实现逻辑问题；单元测试成功但集成测试或接口测试失败，则明确是模块间交互接口的问题；后端接口测试成功，但是UI测试失败，则问题大概率出在前端或前后端交互接口上。

3. 维护成本

上层更接近用户，需求澄清、变化都需要人工确认和调整；下层更贴近AI实现的细节，如果软件设计良好的话，交给AI的修复成功率高，人需要介入的频度可以更低。

"验收测试"与"UI 自动化测试"不是同义词

区分这两个概念

验收测试（Acceptance Test）

关注"需求是否满足"，可跑在 API 层或服务层，只要能验证用户可感知的行为即可。

UI 自动化测试（UI Automation）

一种测试执行手段，关注 UI 交互与呈现。它可以用来实现验收测试，但不必然。

实用原则

能在更低层验证的领域规则，尽量**不要**强行用 UI 去自动验证。**UI 测试其实很昂贵。**

可测试性（Testability）

什么是可测试性？

可测试性是指一个软件系统能否被轻易、快速、稳定地验证对错。很多测试写出来痛苦（例如需要启动一堆服务、准备大量数据才能测一个简单的计算），原因往往不是测试人员能力不行，而是系统本身"不可测"。

在生成式开发中，如何通过提示（Prompt）提升可测试性？（微观控制）

1. 明确分离计算与依赖

"请将金额计算部分独立成一个函数，不要在里面直接调用数据库。"

2. 要求清晰的输出

"请确保该功能执行完毕后，返回一个包含明确状态（如成功/失败原因）的可测试对象，而不是只在控制台打印日志。"

3. 接口解耦

"请将对外部系统（如黑名单系统、邮件服务）的调用抽象出来，以便我们在测试时可以轻易替换为桩服务。"

可以通过从高层需求出发，要求 AI 给出测试用例设计和可运行的验收测试代码。

03

用户故事与验收标准

Making Requirements Verifiable

什么是用户故事?

用户故事描述了对系统或软件的客户或用户有价值的功能片段

- 故事由三方面组成（3C）
  - **卡片（Card）：**一个书面的、简短的故事描述，用来做计划并在开发过程中起到提醒的作用
  - **对话（Conversation）：**针对故事描述进行的交流，用来澄清故事的细节
  - **确认（Confirmation）：**测试，用于表达和归档故事细节且可用于确定故事何时完成



卡片（Card）



交流（Conversation）



确认（Confirmation）

用户故事（User Story）的基本格式与作用

常用格式

作为（As a） X [什么用户角色],
我要（I want） Y [能做什么] ,
以便（so that） Z [达到什么目的或实现什么价值]

- 如果为了更加突出价值，也可以把so that放在前面作为“目的”
  - 为了Z，X要Y
- 如果角色、目的非常明显，也可以省略它们
- “I want”应该是简洁、用户可感知的功能描述

用户故事并没有必须遵循的标准模板；任何能够**准确表达**下述内容的**简单**文字，都是好的方式：

- **谁是用户**（用户类型/角色）
- **用户要完成什么任务**（功能或者动作）
- **用户为什么要这个**（领域价值或者理由）

用户故事卡片的示例

- 在一个**自习座位预定系统**中

作为（As a） 学生用户，
我要（I want） 预约座位，
以便（so that）在我期望的时间能在教室里自习

或者

学生用户可以预约座位

- 对用户角色而言，价值非常清楚，无需多言

交流+确认

- 后续还要对这个用户故事的具体含义进行讨论，确定验收标准（测试）

INVEST：评审故事质量的原则



常用于检查故事是否"写得可做"：

Independent（独立）：故事之间相互独立，避免依赖关系导致的排期阻塞。

Negotiable（可讨论）：故事不是合同，细节可以在实现过程中协商调整。

Valuable（有价值）：每个故事都必须为用户或业务带来明确价值。

Estimable（可估计）：团队能够对故事的工作量进行合理估算。

Small（小）：故事粒度足够小，可以在一个迭代周期内完成。

Testable（**可测试**）：要确定一个完成的标准，用验收测试表述出来。

验收标准 (Acceptance Criteria, AC)

验收标准的目标是：把"满足/不满足"的边界写清楚，让团队在实现前就能对齐。

实用写法

1. 先写主路径（Happy path）

描述正常流程下系统应有的行为。

2. 再补关键约束/异常路径

覆盖权限、校验、并发、错误提示、边界条件等其他情况。

3. 每条 AC 尽量能映射到一条或少数几条自动化测试

确保验收标准是可执行的、可验证的。

- AI 当然可以帮助我们来写这些内容，但人需要审核确认。

实例化需求：通过具体的示例确定验收标准

在实践中，我们往往更擅长描述具体的“事情”。把对具体场景的描述**实例化**并转化为 **Given/When/Then** 格式，就是AC 最精确的表达。

为什么要实例化？（避免"我认为"与"你以为"）

未细化为最终测试描述的验收标准

系统应不允许同一个座位的重复预约。

要考虑哪些具体场景，输入是怎样的，系统行为应该是怎样的（例如输出是什么，给什么错误提示），都应该明确

实例化后的 G/W/T

Given 用户 A 已经在2026/5/10 13:00-14:00预约座位 101
When 用户 B 预约 2026/5/10 13:00-14:00时段的座位 101
Then 系统应该拒绝预约
And 提示**座位已经被其他用户预约**

Given 用户 A 已经在2026/5/10 13:00-14:00预约座位 101
When 用户 A 再次预约 2026/5/10 13:00-15:00时段的座位 101
Then 系统应该拒绝预约
And 提示**座位的预约时段发生冲突**

实例化需求：通过具体的示例确定验收标准

1. 识别关键场景（Scenario）

- 案例 A（主路径）：座位空闲，预约成功。
- 案例 B（异常）：座位已被占用，预约失败。

2. 将案例转化为 Given/When/Then 规格

场景 1：座位空闲时成功预约（主路径）

Given 座位 "S-001" 为空闲状态
When 用户点击"预约座位"
Then 创建预约，座位状态更新为 "已预约"

场景 2：座位已被占用时拒绝预约（异常路径）

Given 座位 "S-002" 已被预约
When 用户点击"预约座位"
Then 系统应拒绝预约并提示 "座位已被占用"
And 座位状态应不变，保持为 "已预约"

仔细看一看，这里你有没有发现什么问题？

实例化需求的作用：

- **对人**：业务人员能看懂“ 判定依据”，发现不符合预期之处
- **对 AI**：AI 能根据 Given 准备数据，根据 When 调用函数，根据 Then 编写待验证内容（断言，assertion）
- **对系统**：这组场景直接固化成了系统的“ 活文档”

案例：自习座位预约系统 SeatReservation

应用：SeatReservation（座位预约 Web 应用）

核心功能点：用户 → 预约座位



Gherkin（Given/When/Then）：可执行规格的桥梁

把讨论从"代码怎么写"拉回到"用户行为与系统可观察结果"。

```
Feature: 预约座位

Scenario: 座位充足时预约成功并通知用户
Given 座位 "S-001" 当前为空闲状态
And 用户 "alice" 的预约中没有座位
When 用户提交预约
Then 系统应创建预约且座位状态为 "已预约"
And 系统应发送一条 "预约成功" 通知给用户 "alice"
```

Gherkin 的价值

- 业务人员、开发人员、测试人员使用同一套语言描述需求
- 场景可直接映射为自动化测试代码
- 需求文档即测试代码，避免文档与代码"两张皮"

Gherkin 异常与边界场景示例

```
Scenario: 座位已被占用时预约失败

Given 座位 "S-001" 当前已被预约
And 用户 "alice" 的预约中没有座位
When 用户提交预约
Then 系统应拒绝预约且提示 "座位已被占用"
```

```
Scenario: 预约超时时预约状态应可解释且允许用户继续操作

# ... 省略 Given 和 When ...
And 预约服务在 30 秒内没有返回明确结果
Then 预约状态应为 "PENDING_CONFIRMATION"
And 用户应看到明确提示 "预约结果确认中"
```

审核要点

"预约超时"到底意味着什么？

- 是否需要复杂的外部审核？异步预约？
- 如何避免"双预约"？
- 超时后的状态对用户是否可解释？

需求不清晰、边界场景不明确往往是 Bug 的高发区，必须在 AC 中明确。

04

自动化验收测试的实现

From Stories to Executable Specifications

自动化验收测试的作用

自动化验收测试更接近**"可执行需求"**。它通常具备：

· **以领域语言命名**：失败时直接对应哪条业务规则没满足。

· **覆盖关键路径**：抓住核心业务价值与风险点。

· **独立稳定运行**：不把随机性、对外部系统依赖当作测试条件。

从**用户故事**到**验收标准**，再到**测试代码**：

- 需要给出系统总体架构设计，例如模块和接口
- AI能帮助做绝大部分的事情
- 验收测试不一定要从前端开始贯穿后端，也可以前后端分别做验收测试
  - 后端更多是接口测试
- 如何不依赖于外部服务或外部系统？

外部服务依赖的测试策略

当我们开发的系统存在外部依赖（比如短信通知、支付服务、物理设备）时，可以将真实的外部依赖替换为 **Stub** 或 **Mock**，这样就可以单独对系统内部的逻辑进行测试。

Stub：替代外部依赖提供一个固定的返回结果（仅提供行为或数据的模拟）。

Mock：不仅可以提供行为和数据的模拟，还可以用于记录和验证行为过程（提供行为模拟）。

为什么需要这些"替身"?

- **隔离依赖**：测试时，外部依赖可能不可用，或者不可靠。
- **模拟外部行为**：Stub 和 Mock 可以模拟外部依赖的行为，确保系统在不同场景下的响应符合预期。
- **验证交互约束**：Mock 可以记录和验证系统与外部依赖的交互，确保符合业务规则。

同一条需求描述的三种实现方式

需求：座位已被占用时应拒绝预约，且不应给用户发通知

它涉及：需求描述（座位占用如何判定）、外部依赖边界（通知服务）、交互约束（"不应发送通知"需要写进 AC）。

接下来我们将展示 Python (pytest) 和 JS/TS (Playwright) 的落地写法。

三种实现方式对比

1. Python + pytest（后端：服务层/组件层代码）

直接在 Python 中编写测试函数，使用 Fake 对象替代外部依赖。

2. Python + Behave（后端：贴近自然语言风格的代码，需要外部框架支持）

使用 Gherkin 语法描述场景，通过步骤定义映射到 Python 代码。

3. JS/TS + Playwright（前端：直接从 GUI 发起验收测试）

通过浏览器自动化验证用户可见的交互结果。

Python + pytest（后端：服务层/组件层代码）

```
# 测试用例：座位已被占用时预约失败且不应发通知
def test_reserve_rejected_when_occupied_and_should_not_notify():
  # Given: 座位 "Seat-101" 当前已被预约状态
  seat_repo = FakeSeatRepository({"Seat-101": "OCCUPIED"})
  notification = NotificationServiceStub()
  svc = ReservationService(seat_repo, notification)
  # When: 用户提交预约请求
  result = svc.reserve(seat_id="Seat-101", user_id="alice")
  # Then: 预约失败且提示"座位已被占用"
  assert result["status"] == "REJECTED"
  assert result["reason"] == "座位已被占用"
  # And: 不应调用通知服务发送短信
  assert notification.sent_messages == []
```

关键设计

- 使用 FakeSeatRepository 和 NotificationServiceStub 隔离外部依赖
- 测试命名直接体现业务语义：test_reserve_rejected_when_occupied_and_should_not_notify
- Given/When/Then 通过注释明确标注，与验收标准对应

Python + Behave（贴近自然语言风格）

reserve.feature

```
Feature: 预约座位

Scenario: 座位已被占用时预约失败
Given 座位 "S-001" 当前状态为 "OCCUPIED"
When 用户 "alice" 提交 "S-001" 的预约请求
Then 系统应拒绝预约且提示 "座位已被占用"
And 系统不应给 "alice" 发送预约成功通知
```



步骤定义 (steps.py)

```python
from behave import given, when, then

@given('座位 "{seat_id}" 当前状态为 "{status}"')
def step_given_seat_status(ctx, seat_id, status):
    ctx.seat_repo = FakeSeatRepository({seat_id: status})
    ctx.notification = NotificationServiceStub()
    ctx.svc = ReservationService(ctx.seat_repo, ctx.notification)

@when('用户 "{user_id}" 提交 "{seat_id}" 的预约请求')
def step_when_reserve(ctx, user_id, seat_id):
    ctx.result = ctx.svc.reserve(seat_id=seat_id, user_id=user_id)

@then('系统应拒绝预约且提示 "{msg}"')
def step_then_rejected(ctx, msg):
    assert ctx.result["status"] == "REJECTED"
    assert ctx.result["reason"] == msg

@then('系统不应给 "{user_id}" 发送预约成功通知')
def step_then_no_notification(ctx, user_id):
    assert user_id not in [n["user_id"] for n in ctx.notification.sent_messages]
```

JS/TS + Playwright（UI 验收风格）

UI 层只需覆盖"用户看到什么提示"，复杂业务判定尽量下沉。

```
import { test, expect } from '@playwright/test';
test('座位已被占用时预约失败并提示用户', async ({ page }) => {
  // Given: 用户在预约页
  await page.goto('http://localhost:3000/reserve/Seat-101');
  // 选择时间段
  await page.getByLabel('预约时间段').selectOption('14:00-16:00');
  // When: 用户提交预约 (使用语义定位器)
  await page.getByRole('button', { name: '确认预约' }).click();
  // Then: 用户看到明确提示，且仍停留在预约页
  await expect(page.getByText('座位已被占用')).toBeVisible();
  await expect(page).toHaveURL(/\/reserve\/Seat-101/);
});
```

关键设计

- 使用语义定位器 getByRole('button', { name: '确认预约' }) 而非 CSS 路径
- expect(...).toBeVisible() 自动等待元素出现，避免硬等时间
- 同时验证提示文本和 URL，确保用户停留在正确的页面

常见失败模式：为什么有些测试会"贵且不稳定"

典型原因（Flakiness 的来源）

- **环境与数据不可控**

共享测试环境、数据被并发修改、依赖外部服务。

- **断言不可观察**

只能看日志、没有稳定的提示/状态。

- **UI 选择器不稳定**

依赖 CSS 层级结构，前端重构即失效。

- **异步等待写法错误**

硬等固定时间，而非等待可观察条件。

应对思路

- 避免依赖不可控的外部服务或系统
- 把测试用例从UI放到服务层/API 层
- 为 UI 增加稳定的测试接口（data-testid）
- 将数据准备做成可复用工厂（Fixture）

用不同测试定位问题在哪个环节

当某个需求验收失败时，用下面的信号快速缩小范围：

- **单元/服务层测试失败**

更可能是领域规则、状态流转、错误处理没按 AC 写清楚。

- **集成测试失败**

更可能是与黑名单/缓存/通知的接口契约、超时/重试语义不一致。

- **UI/E2E 测试失败但低层都通过**

更可能是前端展示逻辑、交互流程、等待条件、选择器稳定性问题。

> Hint：AI 往往会自己发现问题，然后自己尝试修复，但是修复的方式可能与人预期的不一致，并且可能产生大量额外开销。尝试通过 Rule 或 Skills 或提示，要求 AI 先输出"假设列表 + 最小验证实验"，而不是直接重写一大片代码。

05

人机协同测试与开发工作流

Human-AI Collaborative Testing Workflow

结构化思维：前置准备

在开始编写具体的用户故事或实例化需求卡之前，尽量完成对整体需求的**结构化思考**。

为什么要先进行结构化思考？

直接进入细节（如"点击按钮弹出报告"）容易陷入"只见树木不见森林"的困境：

- **遗漏关键环节**

例如只考虑成功生成，忘了数据上传失败或审核不通过的流程。

- **模块边界模糊**

系统复杂性没有合理拆分，模块划分过大导致后续修改困难。

- **缺乏整体视角**

难以维护后续的需求变更。

结构化梳理的三个维度

01
功能模块划分

系统由哪些核心部分组成？

示例：用户中心、座位管理模块、预约规则引擎、通知模块。

02
业务流程梳理

数据和用户操作是如何流转的？

示例：浏览空闲座位 → 发起预约 → 检查预约冲突 → 锁定座位 → 发送成功通知。

03
领域知识整合

这个专业领域有哪些特有的术语、单位和约束规则？

示例：座位预约中的"排他性"、"违约记录"、"签到签退机制"。

AI 可以辅助梳理：让 AI 给出一个结构化梳理草案，人再校对并加入关键意图。

核心概念：增量开发与快速反馈

增量开发 (Incremental Development)

将系统拆解为多个独立、可交付且可测试的微小部分。每一轮增量都在前一轮基础上扩展。

类比：建一个大型游乐场，先做整体布局规划，然后一部分一部分地建造、交付，每次交付都有一些直接可以玩的项目，必要时还可以对已有的部分进行改造重建

价值：降低复杂度、及早交付价值、控制质量风险。

快速反馈 (Fast Feedback)

反馈的代价规律：发现错误越晚，修复成本越高。

- 发现错误的需求，直接修复：接近 0 成本
- 功能完全实现后再发现需求问题：修复成本数十数百倍增加

自动化测试为增量开发提供快速反馈，确保每个改动都能第一时间得到验证。

"大爆炸 (Big-Bang)"：效率与风险的博弈

一次性把需求喂给 AI，让它直接吐出完整代码。

何时适用?

写小规模低复杂度的软件，比如几十行的数据处理脚本、验证想法的抛弃型原型、无严格要求的创意工具。

何时建议使用增量开发（Big-Bang 会变成陷阱）？

当你的软件包含复杂的领域规则、需要长期维护、有严格验收标准时，风险致命：

1. **架构设计失效**：缺乏长远视角，职责不清、耦合严重。
2. **意图匹配偏差**：自然语言歧义导致偏离真实意图。
3. **代码冗余与幻觉**：虚构接口，导致系统异常臃肿。
4. **重构与纠偏困难**：在几千行生成的代码丛林中做"外科手术"。

自动化测试在生成式开发中的价值

在传统开发中，测试往往是"写完代码后的补充"。

在测试先行的开发中，测试是开发的基础，代码是测试的实现。

在生成式开发中，极速的代码生产能力带来的风险：

· 隐性回归与蝴蝶效应

代码生成量大，极易破坏已有稳定逻辑。

· 功能实现偏离需求细节

总体似乎做对了，但在特定边界条件或者细节上总是出错。

· 需求变化难以精确管理

需求总有补充和修改，但难以确定需求是否真的被实现了。

自动化测试：不再盲目祈祷 AI 写对，而是人管住需求和高层的测试，让 AI 自己验收产出。

实践：引导 AI 进行增量开发与持续反馈

以 SeatReservation 自习座位预约系统为例，拒绝"大爆炸"：

- 先引导 AI 实现最核心的"座位预约"逻辑，并配合验收测试确保 Given/When/Then 完全符合预期。
- 在座位预约逻辑稳固后，再引入预约取消、签到等增量。
- 最后引入座位维护等管理功能的增量。

锁定反馈

每步完成后，运行已有验收测试。步步为营，确保对系统的掌控。

驾驭AI开发的基本思路

不要一上来就让 AI 写代码！让AI担任需求分析、架构设计、编码实现多种角色；开发方式由人来确定。

基本方法步骤

1. 人编写总体需求描述：阐述系统背景、目标、功能范围、非功能需求等。

2. AI 协助细化需求分析：针对原始需求描述，明确领域术语，采用用户故事和 G/W/T 的形式逐步细化需求（不实现代码）。

3. 人确认核心需求描述：抓住最关心的核心价值初步确认用户故事和 G/W/T 验收场景，明确迭代实现的优先级。

4. AI(和人一起)根据迭代要求梳理架构：创建项目总体框架，核心模块及其相互关系，并根据用户故事和 G/W/T 验收场景，设计模块之间的接口，基于接口实现用户故事的 G/W/T 验收测试用例。

5. 人确认验收测试用例代码：在相信 AI 和评审 AI 之间找一个合适的平衡，把最核心的需求、模块和接口设计把握好。

6. AI 增量迭代地开发：由 AI 增量迭代地编写代码，必要时利用代码质量控制规则或者 Skill 来提升代码编写的质量。

测试作为增量迭代开发的抓手

当迭代需求确定后，让 AI 以用户故事或接口可验证为标准生成代码。

1. 人类抓总，AI 实现

· 人类精力有限

AI 秒级生成的海量代码和测试，人类无法全量逐行 Review。

· 机器验证机器

先写测试再写实现，让 AI 能在生成时自我验证功能正确性。

2. 增量迭代与检查点（Checkpoints）

· 排优先级

先做核心价值点和主路径，再做支撑性的功能。

· 步步为营

把每个场景或迭代作为检查点，测试通过后才进入下一个场景/迭代。

测试驱动开发方法（TDD）

开发铁律：没有失败的测试，绝不写生产代码。

这能彻底避免 AI 自由发挥、一次性生成导致的不可控局面（Big-Bang）。

红-绿-重构 循环

RED（红）

AI 根据场景先生成测试，**运行并看着它失败**。

GREEN（绿）

AI 编写**最少**的代码让测试通过。

REFACTOR（重构）

在测试全绿的保护下，优化代码结构。

AI 通过提示或调用技能，也可以用这种方法进行开发。代码生成量变大，但是代码质量会提高。

参考 superpowers/skills/test-driven-development/SKILL.md at main · obra/superpowers

掌握合适的交互频度

交互频度应当与软件的复杂度、开发目的相匹配：

极低频度（Big-Bang）

适用简单脚本、抛弃型原型。一次性丢给 AI，错了重开。需要大量的前期准备（比如准备详细spec）。

高频度（微观管理）

适用局部的关键代码或核心算法，但需要定义良好的边界。有时可能 AI 会意外干很多事情。

中等频度（增量迭代模式）

以合适的粒度交付功能场景，逐步完善系统的各个部分，并且每个部分都有测试来保护。

参考人类完成功能的粒度：以"一个验收场景（Scenario）"为周期。

要求 AI 按照既定的用户故事和 G/W/T 场景，一个一个端到端完成；人确认后，再进入下一个。系统始终可见、可运行、可测试。

持续演进和小步增量修改

关键需求是否都有对应的自动化测试保护

可以拆解为三类缺口：

· 需求缺口

某条验收标准没有任何自动化测试来验证。

· 层级缺口

本应在服务/API 层验证的规则，被放在 UI 层导致脆弱。

· 变更缺口

代码变了，但相关用例没有更新或新增。

应对持续演进：

· 把测试与用户故事或验收场景对齐

例如要求 AI 生成测试代码时同时生成函数级注释，表明对应哪个场景，便于人和AI验证。

· 需求变更了怎么办？

同步更新验收测试场景和验收测试代码。

Tip: 让 AI 分析需求或设计的变化（生成变化的自然语言描述，语义 diff），然后让 AI 基于语义 diff 进行最小化修改。

· 定期清理"无价值测试"

跑得慢、断言弱、与实现强耦合的测试应该被重构或删除。

演示：自习座位预约系统的开发

Step 1: 原始需求编写

人：写出对系统总体的要求

Step 2: 用户故事编写

AI 基于人的描述，输出用户故事

人：Review 并排定优先级："先做座位预约和座位查询的功能"

Step 3: 细化用户故事的验收测试标准

AI 生成 Gherkin 描述的验收测试标准

人：Review 测试用例，确认是否符合需求

Step 4: 架构设计与接口实现

AI 按照迭代要求，给出相应架构和接口设计

人：Review 架构和接口设计

Step 5: 增量开发，人工确认

AI 开始迭代开发

人：关注验收测试、关键接口测试的完成情况，推敲需求细节

按照迭代循环继续，直到所有用户故事完成……

06

界面自动化测试入门

UI Automation with Playwright

什么时候值得做 UI 自动化

UI 自动化很贵，先问三个问题：

1. 这条路径是否足够关键?

高频/高风险/影响收入或安全？

2. 这条路径是否只能通过 UI 验证?

交互、渲染、端到端集成是否必须走 UI？

3. 页面与组件是否具备稳定可测性?

可用 role/label/testid 可靠定位？

如果答案都偏"是"，UI 自动化才更划算。

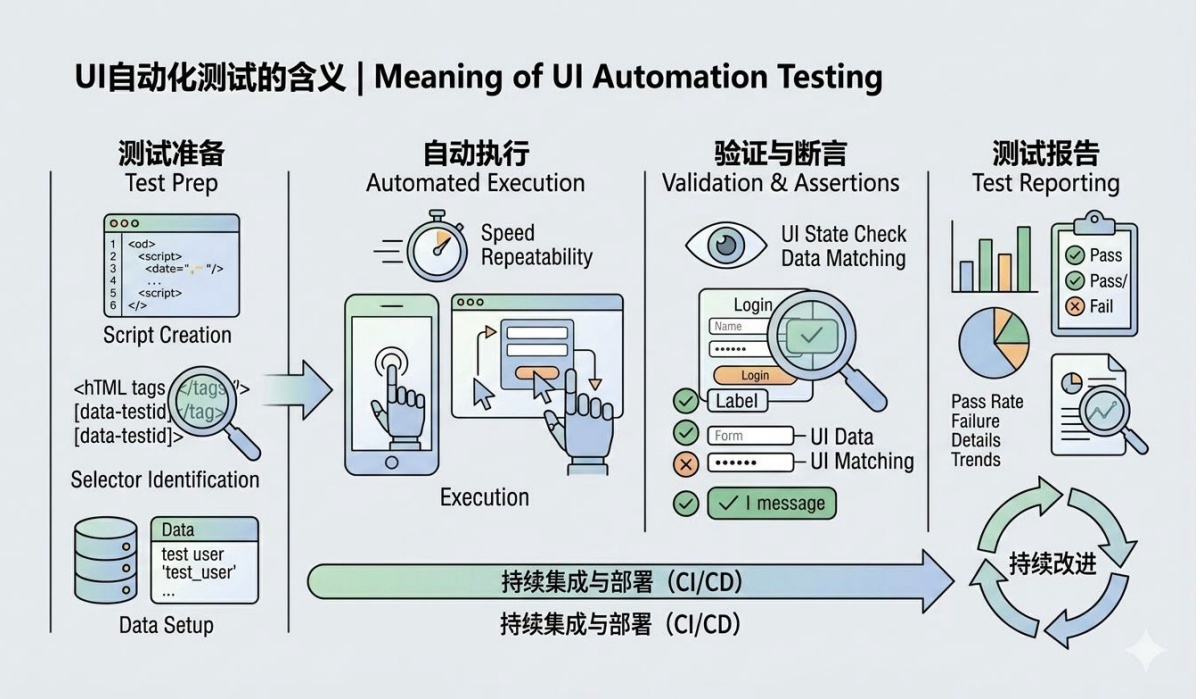


Fig by Gemini

Playwright 核心概念与基础示例

核心概念（基本版）

page：浏览器页面对象，代表一个标签页。

locator：用于定位元素，"懒执行"的，具备自动重试与等待能力。

expect：断言并触发自动等待，直到条件满足或超时。

test：测试用例定义；可用 beforeEach 做公共准备。

基础使用示例

```
import { test, expect } from '@playwright/test';
test('用户登录成功后应跳转到仪表盘',
async ({ page }) => {
await page.goto('https://example.com/login');
await page.getByLabel('用户名').fill('alice');
await page.getByLabel('密码').fill('secret123');
await page.getByRole('button',
{ name: '登录' }).click();
await expect(page.getByRole('heading',
{ name: '欢迎, alice' })).toBeVisible();
await expect(page).toHaveURL(/.*dashboard/);
});
```

写稳定 UI 测试的关键原则与语义定位器

四原则

1. 用语义定位器：优先 getByRole/getByLabel/getByText，其次 data-testid
2. 避免硬等待：尽量用 expect 等待可观察状态，不用 sleep
3. 断言"用户看得见的结果"：提示、跳转、关键文本、按钮状态
4. 把准备数据下沉：通过 API/后门/fixture 创建数据，不要全靠 UI 狂点

语义定位器示例对比

```
// ❌ 脆弱的定位器
await page.locator('#app > div > div.content > form > button.btn-primary').click();
```

```
// ✅ 稳定的语义定位器
await page.getByRole('button', { name: '提交' }).click();
```

```
// ✅ 如果没有合适的语义标签，使用专属的测试 ID
await page.getByTestId('submit-order-btn').click();
```

常见反例（不稳定 Flakiness）

- **依赖脆弱选择器**

.btn > span:nth-child(2)

- **依赖固定时间**

await page.waitForTimeout(3000)

- **断言过于实现细节**

断言某个 div 的嵌套层级深度。

一旦前端修改样式或逻辑，这些测试立刻全部失效。

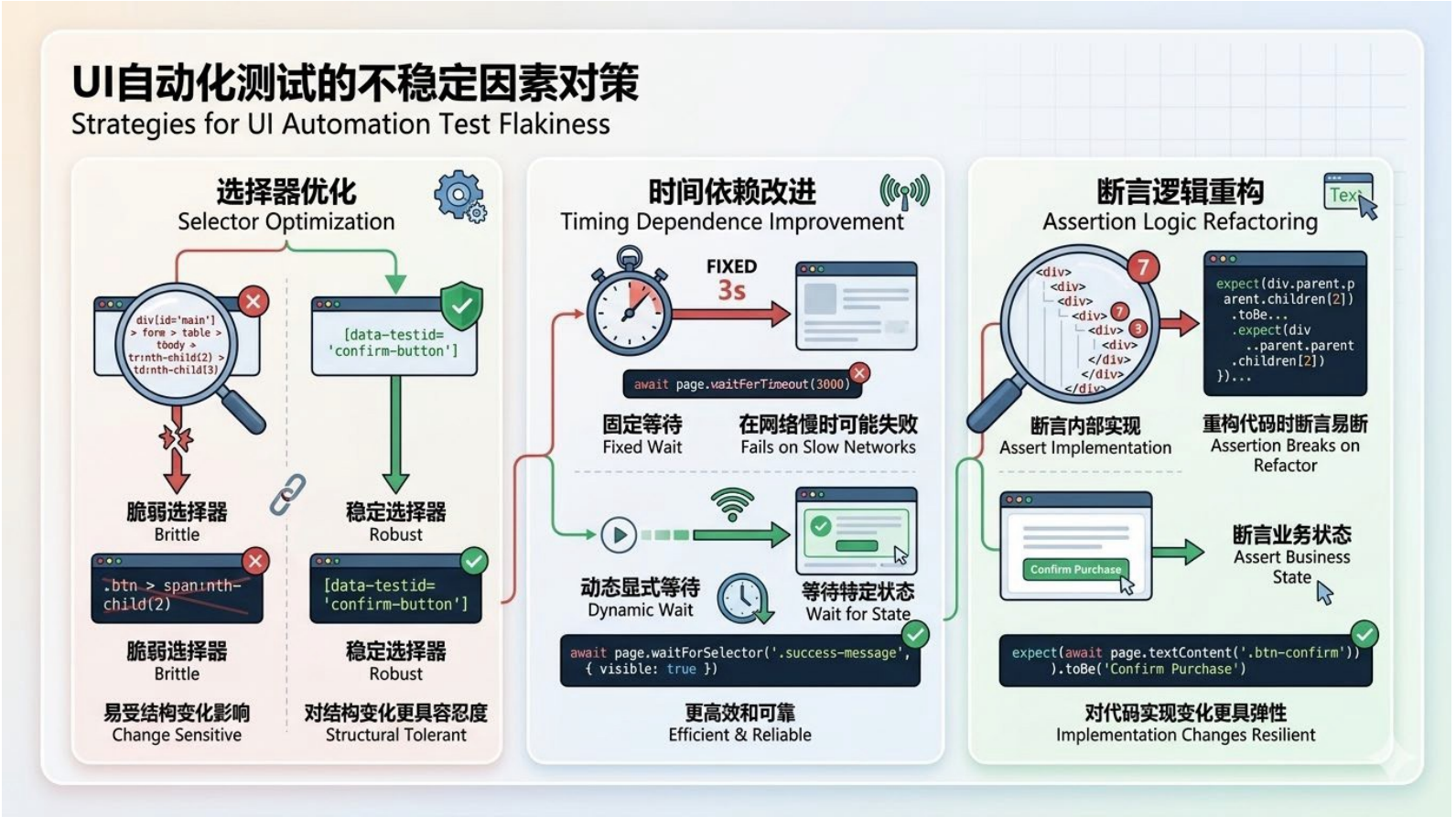


Fig by Gemini

07

练习与讨论

Exercise & Disccusion

练习 与 讨论

选择自习座位预约系统或者你想做的一个任何系统，考虑如何：

A

把需求写到"可验收"

写 1-2 条用户故事及 5-8 条 AC，将其中 2-3 条改写为 Gherkin 场景。

B

把场景落地成自动化验收测试

选 1 个场景，用 Python/Java/TS 落地。要求：断言对应 Then，命名体现领域语义。

C

用例评审（互评）

指出对方最明显的"需求缺口"与"不稳定 (Flakiness)"的点。

生成式软件开发

End

8. 自动化测试